PROJECT OVERVIEW

Dompet Rumah

A household finance app for couples: salaries, prorated THR and bonuses, recurring bills, monthly plans, savings goals, and projections in one place, shared from two phones.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

ROLE

Personal project · design, frontend, database schema & RLS, deployment

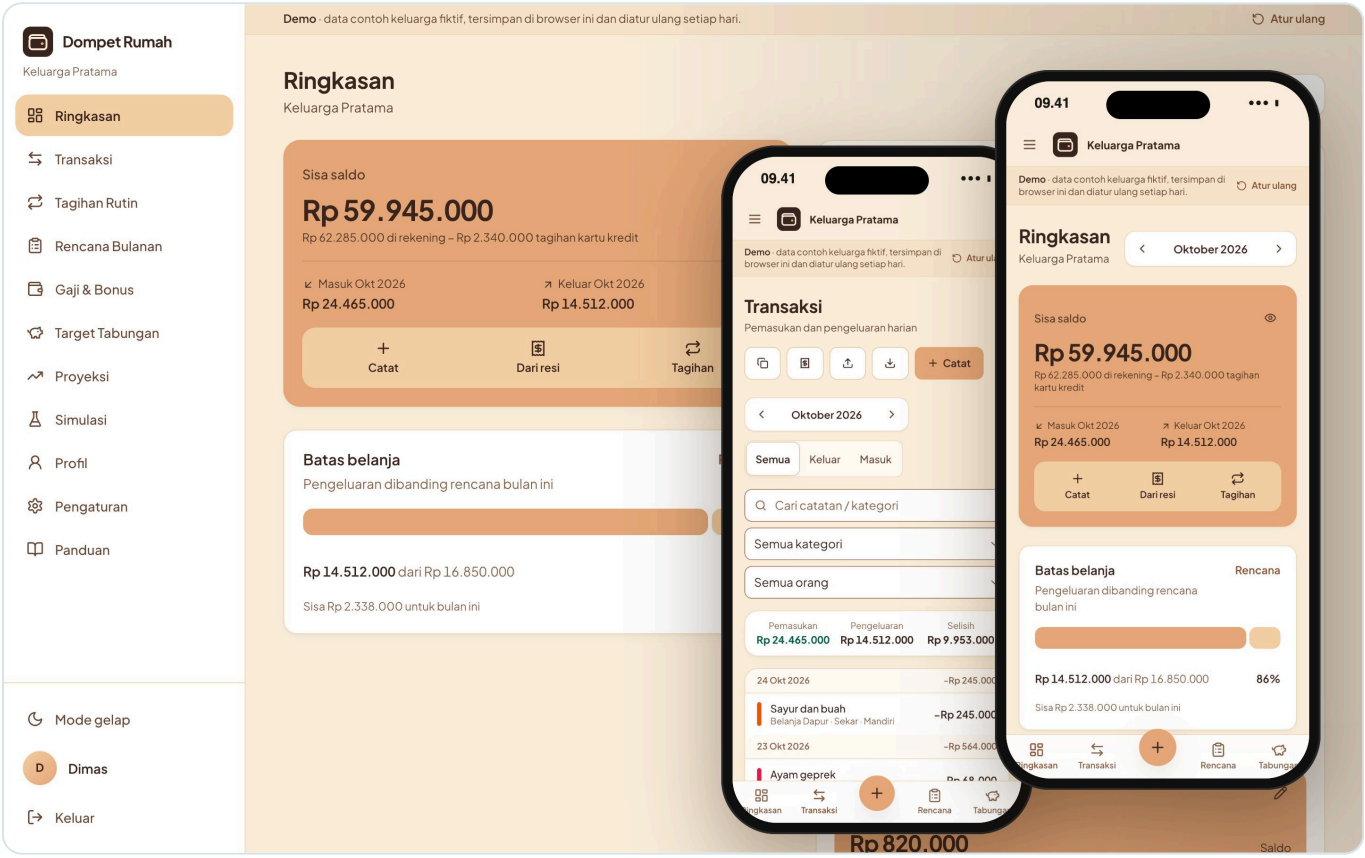
PLATFORMS

Web (desktop & mobile), installable as a PWA

LINKS

Live demo: raihan-evanza-dompet-rumah.vercel.app

React 18 · TypeScript · Vite · TanStack Query · Supabase (Postgres 17, Auth, Realtime) · Row Level Security · Tailwind CSS · Rupa Design System · Recharts · pdf.js + Tesseract.js · SheetJS · Vitest



OVERVIEW

Household money is usually scattered: two salaries paid on different days, THR and bonuses prorated for whoever changed jobs, recurring bills due across the month, and balances spread over bank accounts, e-wallets, and a credit card. A shared spreadsheet gets messy fast and is hard to fill in from a phone.

Dompet Rumah brings it together for one household. Each person has their own account and a partner joins with an invite code, after which both see the same data: transactions, bills ticked off when paid, a monthly budget per category, savings goals, and a 1–3 year balance projection that counts THR and bonuses in the month they are paid.

The app is a React frontend that talks to Supabase directly. Because the database is reached from the browser, every table is guarded by Row Level Security, and the policies are tested automatically. The public demo is a separate build that keeps all data in the browser, with a fictional family as sample data.

KEY FEATURES

Overview and remaining balance

The remaining balance across bank accounts, e-wallets, cash, and investments minus the credit card bill; this month's spending against the plan; six months of cash flow; spending by category; unpaid bills; goal progress; and the next THR or bonus.

Transactions and import

Income and expenses per month, filtered by category, person, or search. Copy from last month, import CSV/Excel including bank statements with Debit/Credit columns (duplicates detected), and export to CSV/Excel.

Record from a receipt

A PDF or screenshot of a payment receipt is read in the browser (pdf.js for PDFs, Tesseract for images) without being uploaded. The date, amount, and recipient are filled in, and the account is recognised from the bank name or the last four digits.

Recurring bills

Mortgage, instalments, electricity, internet, school fees, and more, each with a due day. Pressing Pay creates the transaction, and the database refuses the same bill being recorded twice in one month.

Salary, THR, and prorated bonuses

Net salary and base pay per person, including raises and previous jobs. THR follows Permenaker 6/2016 (under 12 months of service → months worked/12); annual bonuses can be a salary multiple or a fixed amount, prorated per month or per day, with an estimated tax deduction. Salaries can be recorded automatically on payday.

Monthly plan

A budget per category and an allocation to each savings goal, compared with actual spending and expected income. It can be filled from recurring bills, copied from last month, and imported or exported as Excel.

Savings goals and projection

Deposits and withdrawals per goal, the monthly amount needed (using the annuity formula when there is a return), and an estimated completion date. The 1–3 year projection computes the balance month by month; the Simulation page tries scenarios such as a car down payment and instalments without touching real data.

Accounts and Sync

Account balances are typed in by hand and show up on the partner's phone right away (Supabase Realtime). Transactions tagged with an account are shown as "not yet in the balance", and the Sync button adds them.

ARCHITECTURE & DATA FLOW

Browser (React + PWA)
Pages, finance formulas, and the receipt reader run on the device.

→

TanStack Query
A cache per household; every change refreshes the related tables.

→

Supabase Auth + PostgREST
Per-person sign-in; direct queries with the public (anon) key.

→

Postgres 17 + RLS
Per-table policies, unique constraints, and RPC functions (copy_plan, sync_account).

- There is no application server: the browser calls Supabase directly and only the public key is in the frontend. All access rules are enforced by the database through Row Level Security.
- Rules that must never break (a bill once a month, one automatic salary per month, categories from the same household) are enforced by constraints and composite foreign keys, not only in the UI.
- The THR, bonus, goal, and projection formulas are pure functions in src/lib/finance with unit tests, so the app and the demo compute the same results.

STACK BY LAYER

Frontend	React 18 · TypeScript (strict) · Vite 5 · React Router 6 · TanStack Query 5
UI	Rupa Design System (brand latte) · Tailwind CSS 3 · Recharts · lucide-react
Data & auth	Supabase (Postgres 17, Auth, Realtime) · Row Level Security · Fungsi RPC PL/pgSQL
Files	pdf.js · Tesseract.js · SheetJS · Papa Parse
Quality	Vitest (98 unit test) · Uji RLS di Postgres 17 sementara (Docker) · ESLint · Prettier · GitHub Actions
Deployment	Vercel · Supabase (region Singapura)

TECHNICAL DECISIONS

1. **No backend, security in the database**
   For a family app, a server of its own only adds cost and upkeep. The browser calls Supabase directly, and every table is protected by RLS: an account can only read and change its own household's data. The policies are tested automatically, covering outsiders, regular members, and a created_by that cannot be forged.

2. **Finance formulas as pure functions**
   THR, prorated bonuses, goal deposits, and projections are side-effect-free functions with dates as YYYY-MM-DD strings, so nothing shifts with the time zone and edge cases are easy to test (starting mid-month, leap years, the THR cut-off).

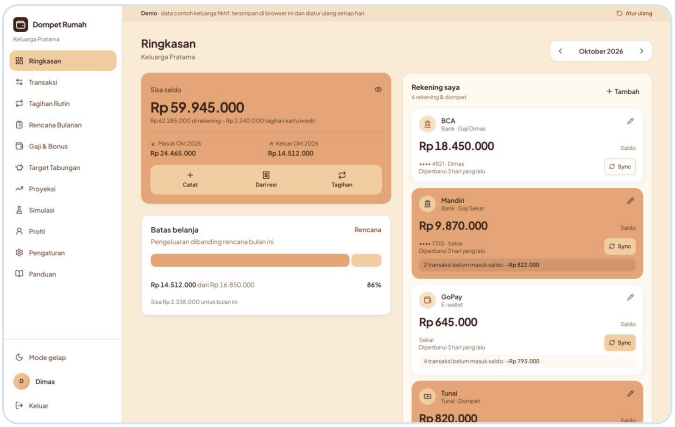
3. **Receipts read on the device**
   Receipts contain account numbers and names. Instead of uploading them to an OCR service, PDFs are read with pdf.js and images with Tesseract in the browser, and the result is always reviewed before saving.

4. **Balances typed in, not computed**
   A balance computed from transactions drifts as soon as something is not recorded. Account balances are typed in as the bank app shows them; transactions recorded afterwards are shown as a pending difference and only added on Sync.

5. **A public demo without real data**
   The real app holds a family's finances, so the demo is a separate build (npm run build:demo): the Supabase client is swapped for an in-browser one that keeps data in localStorage, seeded with a fictional family whose dates follow today and reset daily. No Supabase keys are in the demo bundle.

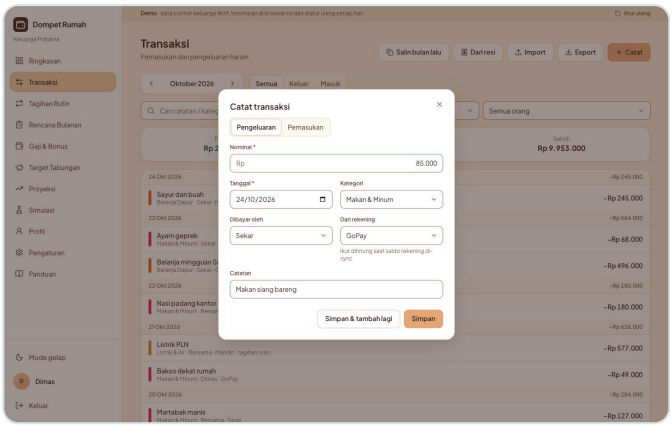
HOW TO USE IT

These steps follow the same order as the tutorial video.



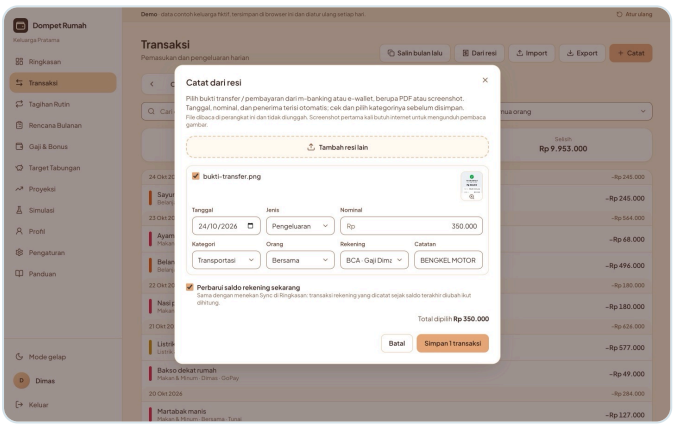
1 Open the overview

The demo signs you in as Dimas of the Pratama family. The overview shows the remaining balance, this month's spending limit, and the accounts. An account with transactions "not yet in the balance" can be synced to update it.



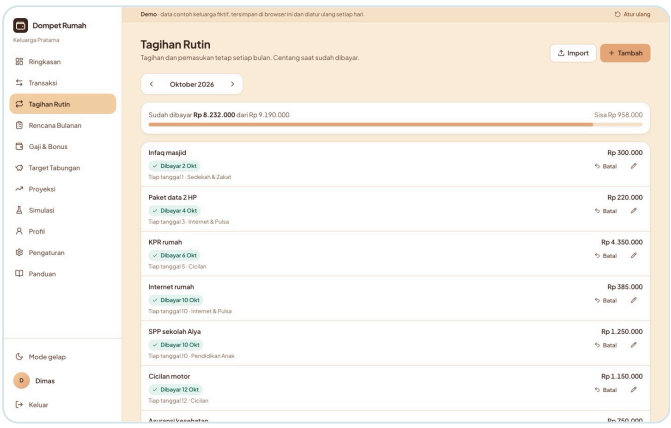
2 Record a transaction

Click Catat (Record), fill in the amount, category, who paid, the account, and a note, then save. On a phone, the + button in the bottom bar opens the same form.



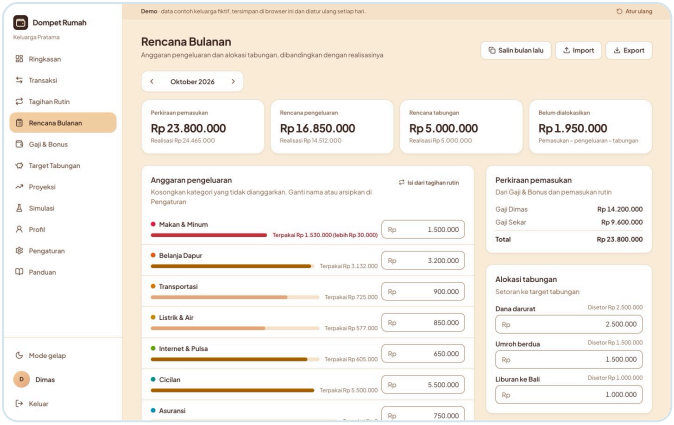
3 Record from a receipt

On Transaksi, click Dari resi (From receipt) and pick a PDF or screenshot. The date, amount, note, and account are filled in; choose a category and save. The first use downloads the image reader.



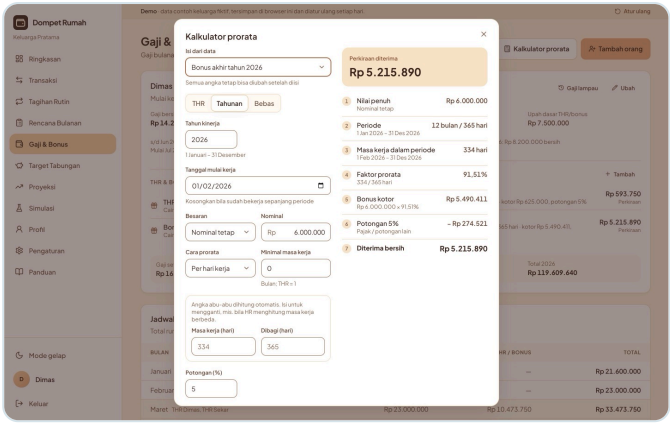
4 Pay a recurring bill

On Tagihan Rutin (Recurring bills), overdue bills are marked red. Press Bayar (Pay), check the amount and date, then save: the bill is marked paid and its transaction is recorded.



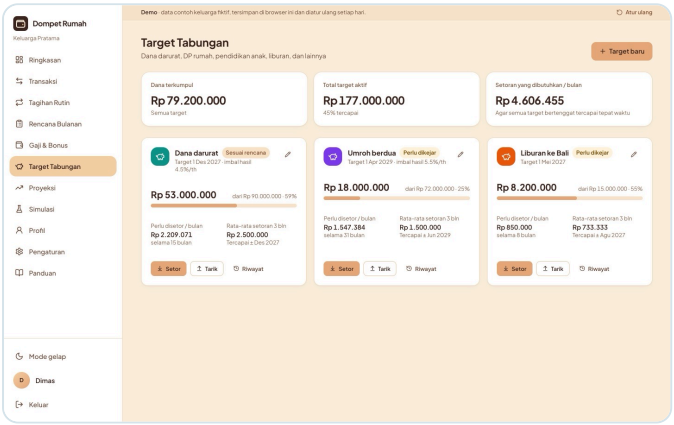
5 Compare the plan with actuals

Rencana Bulanan (Monthly plan) shows each category's budget with an actuals bar; categories over budget turn red. On the right are expected income and the allocation to each savings goal.



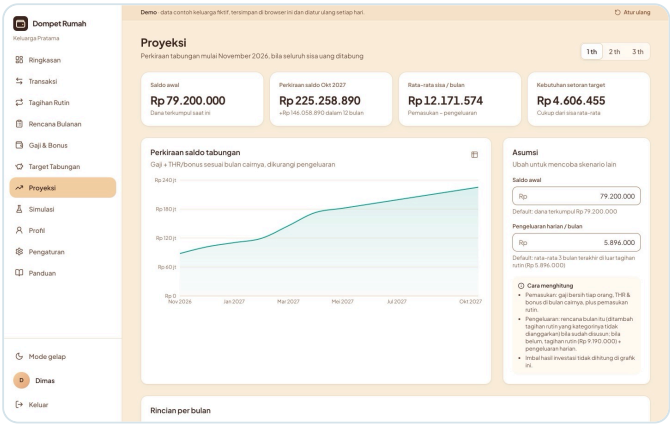
6 Work out THR and bonuses

Gaji & Bonus shows each person's salary, THR, and bonuses plus a yearly income schedule. The proration calculator can be filled from the data, for example Sekar's year-end bonus after starting in February, and shows each step of the calculation.



7 Track savings goals

Each goal shows its progress, the monthly deposit needed, the three-month average, and an estimated completion date. Setor and Tarik record deposits and withdrawals.



8 See the projection

Proyeksi (Projection) computes the savings balance month by month for 1, 2, or 3 years from salaries, THR/bonuses in the month they are paid, and spending. The starting balance and daily spending can be changed to try other scenarios.

LIMITATIONS & NOTES

- The live demo runs without Supabase: sample data lives in each visitor's browser localStorage and resets every day. Sign-in, partner invites, and Realtime between phones exist only in the real app.
- The real app uses Supabase's free tier, which pauses after about a week without use; data is kept, but the project has to be restored from the dashboard. Supabase's built-in confirmation emails are also limited to a few per hour.
- Receipt reading relies on common receipt patterns; unusual layouts can be misread, which is why every result is reviewed before saving.
- There is no automatic bank sync: balances and statements come in by typing, receipts, or file import.

WHAT'S NEXT

- Automatic monthly recaps and bill reminders through push notifications.
- Receipt attachments stored with their transactions (Supabase Storage with per-household policies).
- A yearly report to prepare the annual tax return.